

Tema 3

El Modelo Relacional

1. [Introducción](#)
2. [Objetivos](#)
3. [Componentes de las bases de datos relacionales](#)
 - Relación o tabla: atributo, tupla
 - Dominio
 - Grado
 - Cardinalidad
 - Propiedades de las tablas (relaciones)
 - Claves: clave candidata, clave primaria, clave alternativa, clave foránea, externa, ajena o secundaria
 - Nulos
 - Restricciones
4. [Diseño lógico: paso del modelo entidad/relación al modelo relacional](#)
 - Introducción a la transformación de entidades e interrelaciones
 - A) Transformación de entidades
 - B) Transformación de relaciones binarias
 - B.1) Cardinalidad 1:1
 - B.2) Cardinalidad 1:N
 - B.3) Cardinalidad N:N
 - C) Transformación de interrelaciones ternarias
 - C.1) Cardinalidad N:N:N
 - C.2) Cardinalidad N:N:1
 - C.3) Cardinalidad N:1:1
 - C.4) Cardinalidad 1:1:1
 - D) Transformación de relaciones n-arias
 - E) Transformación de relaciones recursivas
 - F) Transformación de entidades débiles
 - G) Transformación de la generalización/especialización
5. [Ejemplo completo de paso del modelo entidad/relación al modelo relacional](#)
6. [Anexo: las 12 reglas de Codd](#)

1. Introducción

Edgar Frank Codd definió las bases del modelo relacional a finales de los 60. En 1970 publicó el documento *"A Relational Model of data for Large Shared Data Banks"* (*"Un modelo relacional de datos para grandes bancos de datos compartidos"*). Actualmente se considera que éste es uno de los documentos más influyentes de toda la historia de la informática. Lo es porque en él se definieron las bases del llamado Modelo Relacional de Bases de Datos. Anteriormente el único modelo teórico estandarizado era el Codasyl, que se utilizó masivamente en los años 70 como paradigma del modelo en red de bases de datos.

Codd se apoya en los trabajos de los matemáticos Cantor y Childs (cuya teoría de conjuntos es la verdadera base del modelo relacional). Según Codd los datos se agrupan en **relaciones** (actualmente llamadas **tablas**), que es un concepto que se refiere a la estructura que aglutina datos referidos a una misma entidad de forma independiente respecto a su almacenamiento físico.

Lo que Codd intentaba fundamentalmente era evitar que los usuarios de la base de datos tuvieran que verse obligados a aprender los entresijos internos del sistema. Pretendía que los usuarios trabajaran de forma sencilla e independiente del funcionamiento físico de la base de datos en sí. Fue un enfoque revolucionario.

Aunque trabajaba para IBM, esta empresa no recibió de buen grado sus teorías (de hecho, continuó trabajando en su modelo en red IMS) y fueron otras empresas (en especial Oracle) las que implementaron sus teorías. Pocos años después, el modelo se empezó a utilizar cada vez más, hasta finalmente ser el modelo de bases de datos más popular. Hoy en día casi todas las bases de datos siguen este modelo.

2. Objetivos

Codd perseguía estos objetivos con su modelo:

- **Independencia física.** La forma de almacenar los datos no debe influir en su manipulación lógica. Si la forma de almacenar los datos cambia, los usuarios no tienen siquiera que percibirlo y seguirán trabajando de la misma forma con la base de datos. Esto permite que los usuarios se concentren en qué quieren consultar en la base de datos y no en cómo está realizada la misma.
- **Independencia lógica.** Las aplicaciones que utilizan la base de datos no deben ser modificadas porque se modifiquen elementos de la base de datos. Es decir, añadir, borrar y suprimir datos, no influye en las vistas de los usuarios. De una manera más precisa, gracias a esta independencia, el esquema externo de la base de datos es realmente independiente del modelo lógico.
- **Flexibilidad.** La base de datos debe ofrecer fácilmente distintas vistas en función de los usuarios y aplicaciones.
- **Uniformidad.** Las estructuras lógicas siempre deben tener una única forma conceptual (las tablas).
- **Sencillez.** Facilidad de manejo (algo cuestionable, pero ciertamente verdadero si comparamos con los sistemas gestores de bases de datos anteriores a este modelo).

3. Componentes de las bases de datos relacionales

Relación o tabla: atributo, tupla

Según el modelo relacional (desde que Codd lo enunció) el elemento fundamental es lo que se conoce como **relación**, aunque más habitualmente se le llama **tabla** (o también array o matriz). Codd definió las relaciones utilizando un lenguaje matemático, pero se pueden asociar a la idea de tabla (de filas y columnas) ya que es más fácil de entender.

No hay que confundir la idea de relación según el modelo de **Codd**, con lo que significa una relación en el modelo Entidad/Relación de **Chen**. No tienen nada que ver.

Las relaciones constan de:

- **Atributos.** Referido a cada propiedad de los datos que se almacenan en la relación (nombre, dni...). Cada columna de la tabla.
- **Tuplas.** Referido a cada elemento de la relación. Por ejemplo, si una relación almacena personas, una tupla representaría a una persona en concreto. Cada fila de la tabla.

Puesto que una relación se representa como una tabla; podemos entender, como ya se ha dicho, que las columnas de la tabla son los atributos; y las filas, las tuplas.

atributo 1	atributo 2	atributo 3		atributo n	
valor 1,1	valor 1,2	valor 1,3		valor 1,n	← tupla 1
valor 2,1	valor 2,2	valor 2,3		valor 2,n	← tupla 2
.....					
valor m,1	valor m,2	valor m,3		valor m,n	← tupla m

Cada una de las filas de la relación se corresponde con la idea clásica de **registro**, representando, por tanto, cada elemento individual de esa relación. Tiene que cumplir que:

- Cada tupla se debe corresponder con un elemento del mundo real.
- No puede haber dos tuplas iguales (con todos los valores iguales).

Dominio

Un dominio contiene todos los posibles valores que puede tomar un determinado atributo. Dos atributos distintos pueden tener el mismo dominio.

Un dominio en realidad es un conjunto finito de valores del mismo tipo. A los dominios se les puede asignar un nombre y así podemos referirnos a ese nombre en más de un atributo.

La forma de indicar el **contenido de un dominio** se puede hacer utilizando dos posibles técnicas:

- Intensión. Se define el dominio indicando la definición exacta de sus posibles valores. Por intención se puede definir el dominio de edades de los trabajadores como: *números enteros entre el 16 y el 65* (un trabajador sólo podría tener una edad entre 16 y 65 años).
- Extensión. Se indican todos los valores, o algunos y se sobreentiende el resto gracias a que se autodefinen con los anteriores. Por ejemplo, el dominio localidad se podría definir por extensión así: *Salamanca, Burgos, Logroño...*

Grado

Indica el tamaño de una relación en base al número de columnas (atributos) de la misma. Lógicamente cuanto mayor es el grado de una relación, mayor es su complejidad para manejarla.

Cardinalidad

Número de tuplas de una relación, o número de filas de una tabla.

Los términos vistos anteriormente tienen distintos sinónimos según la nomenclatura utilizada. A ese respecto, se utilizan tres nomenclaturas:

Términos 1 (nomenclatura relacional)		Términos 2 (nomenclatura tabla)		Términos 3 (nomenclatura ficheros)
relación	=	tabla	=	fichero
tupla	=	fila	=	registro
atributo	=	columna	=	campo
grado	=	nº de columnas	=	nº de campos
cardinalidad	=	nº de filas	=	nº de registros

A la hora de **especificar una relación**, se puede identificar a través de su esquema o de su cuerpo:

Clientes		
DNI	Nombre	Edad
12333944C	Ana	52
12374678G	Eva	27
28238232H	Martín	33

Esquema: Cliente (DNI, Nombre, Edad)

Cuerpo: {(DNI: "12333944C", Nombre: "Ana", Edad:52), (DNI: "12374678G", Nombre: "Eva", Edad:52), (DNI: "28238232H", Nombre:"Martín",Edad:33)}

Propiedades de las tablas (relaciones)

- Cada tabla tiene un nombre distinto.
- Cada atributo de la tabla toma un solo valor en cada tupla.
- Cada atributo tiene un nombre distinto en una tabla (aunque puede coincidir en tablas distintas).
- Cada tupla es única (no hay tuplas duplicadas).
- El orden de los atributos no importa.
- El orden de las tuplas no importa.

Claves

Clave candidata

Conjunto de atributos que identifican unívoca y mínimamente (no puede sobrar ningún atributo) cada tupla de la relación. Es decir, son las columnas cuyos valores no se repiten en ninguna otra tupla de esa tabla. Toda tabla en el modelo relacional debe tener al menos una clave candidata (puede incluso haber más).

Clave primaria

Clave candidata que se escoge como **identificador** de las tuplas. Se elige como primaria la candidata que identifique mejor a cada tupla en el contexto de la base de datos. No puede tomar valores nulos.

Por ejemplo, un campo con el *DNI* sería clave candidata de una tabla de clientes; si esa tabla tiene también un campo de *código de cliente*, éste sería mejor candidato (y, por lo tanto, sería la clave principal) porque es mejor identificador para ese contexto particular.

Clave alternativa

Cualquier clave candidata que no sea primaria.

Clave foránea, externa, ajena o secundaria

Son los atributos de una tabla cuyos valores están relacionados con atributos de otra tabla. Se especificarán con una flecha hacia arriba ↑. Por ejemplo, en la tabla equipos tenemos estos datos:

Equipo	Nº Equipo
Real Madrid	1
F.C. Barcelona	2
Athletic Bilbao	3

En la tabla anterior, consideramos como clave principal al atributo *nº de equipo*. En otra tabla podríamos tener:

Nº Jugador	Jugador	Nº Equipo
1	Williams	3
2	Messi	2
3	Ramos	1
4	Vinicius	1

El atributo *Nº Equipo* sirve para relacionar el Jugador con el equipo al que pertenece. Ese campo en la tabla de jugadores es una clave externa.

Nulos

En los lenguajes de programación se utiliza el valor nulo para reflejar que un identificador (una variable, un objeto...) no tiene ningún contenido. Por ejemplo, cuando un puntero en lenguaje C señala a *null* se dice que no está señalando a nadie. Al programar en esos lenguajes, nulo se trata de un valor que no permite utilizarse en operaciones aritméticas o lógicas.

Las bases de datos relacionales permiten más posibilidades para el valor nulo (null), aunque su significado no cambia: valor vacío. En las bases de datos se utiliza para diversos fines. Por ejemplo, en claves secundarias indican que el registro actual no está relacionado con ninguno. En otros atributos indica que la tupla en cuestión carece de dicho atributo: por ejemplo, en una tabla de *personas* un valor nulo en el atributo *teléfono* indicaría que dicha persona no tiene teléfono. Es importante indicar que en un campo texto, el texto en blanco ' ', no significa lo mismo que el nulo; como tampoco en un campo numérico el valor cero significaría lo mismo que nulo.

Puesto que es un valor posible que se utiliza continuamente, resulta imprescindible saber cómo actúa cuando se emplean operaciones lógicas sobre ese valor. Eso significa definir un tercer valor en la lógica booleana, además de los clásicos verdadero y falso. Un valor nulo no es ni verdadero ni falso (se suele interpretar usando la aritmética clásica en valores lógicos, el 1 es verdadero, el 0 falso y el 0,5 nulo).

El uso de operadores lógicos con el nulo da lugar a que:

- *verdadero Y (AND) nulo* da como resultado nulo (siguiendo la aritmética planteada antes: $1 \cdot 0,5 = 0,5$).
- *falso Y (AND) nulo* da como resultado falso ($0 \cdot 0,5 = 0$).
- *verdadero O (OR) nulo* da como resultado, verdadero ($1 + 0,5 > 1$).
- *falso O (OR) nulo* da como resultado nulo ($0 + 0,5 = 0,5$).
- *la negación de nulo*, da como resultado nulo.

En todas las bases relacionales se utiliza un operador llamado es nulo (is null) que devuelve verdadero si el valor con el que se compara es nulo.

Restricciones (constraints)

Se trata de condiciones de obligado cumplimiento por las tuplas de la base de datos.

Se pueden diferenciar dos grandes tipos:

- Inherentes: son aquellas que no son determinadas por los usuarios, sino que son definidas por el hecho de que la base de datos sea relacional. Las más importantes son:
 - No puede haber dos tuplas iguales.
 - El orden de las tuplas no es significativo.
 - El orden de los atributos no es significativo.
 - Cada atributo sólo puede tomar un valor en el dominio en el que está inscrito.
- Semánticas: el modelo relacional permite a los usuarios incorporar restricciones personales a los datos. Las más importantes son: clave primaria, unicidad, obligatoriedad, integridad referencial, regla de validación o verificación, disparadores o triggers y representación abreviada de las tablas en el modelo relacional. A continuación, se ven en detalle.

Clave principal (primary key). También llamada clave primaria. Marca uno o más atributos como identificadores de la tabla. De esta forma, en esos atributos las filas de la tabla no podrán repetir valores ni tampoco dejarlos vacíos.

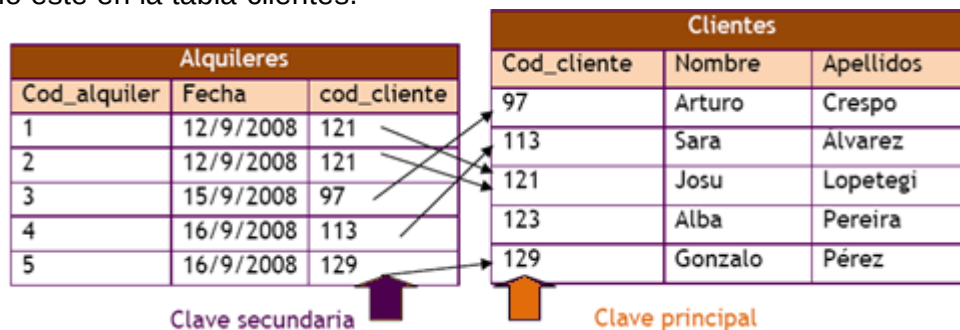
Unicidad (unique). Impide que los valores de los atributos marcados de esa forma puedan repetirse. Esta restricción debe indicarse en todas las claves alternativas. Al marcar una clave primaria, se añade automáticamente un criterio de unicidad sobre los atributos que forman la clave.

Obligatoriedad (not null). Prohíbe que el atributo marcado de esta forma quede vacío (es decir impide que pueda contener el valor nulo, *null*).

Integridad referencial (foreign key). Sirve para indicar una clave externa (también llamada secundaria o foránea) sobre uno o más atributos. Los atributos marcados de esta forma sólo podrán contener valores que estén relacionados con la clave principal de la tabla que relacionan (llamada tabla principal). En principio, estos

atributos contendrán los valores con los que se relacionan de la tabla principal, pero puede llegar a suceder que estos atributos terminen conteniendo valores nulos.

Por ejemplo, si hay una tabla de alquileres en la que cada fila es un *alquiler*, existirá un atributo *cod_cliente* que indicará el código del cliente y que estará relacionado con una tabla de *clientes*, en la que dicho atributo es la clave principal. De hecho, para cumplir con la integridad referencial, no se podrá incluir un código que no esté en la tabla clientes.



Esta situación puede causar problemas en las operaciones de borrado y modificación de registros, ya que si se ejecutan esas operaciones sobre la tabla principal (si se modifica o borra un cliente) quedarán filas en la tabla secundaria con la clave externa haciendo referencia a un valor que ya no existe en la tabla principal. Para solventar este posible problema se puede hacer uso de estas opciones:

- *Prohibir la operación (no action = restrict).*
- *Transmitir la operación en cascada (cascade).* Es decir, si se modifica o borra un cliente; también se modificarán o borrarán los alquileres relacionados con él.
- *Colocar nulos (set null).* Las referencias al cliente en la tabla de alquileres se colocan como nulos (es decir, alquileres sin cliente).
- *Usar el valor por defecto (set default).* Se coloca un valor por defecto en las claves externas relacionadas. Este valor se indica al crear la tabla (opción default).

Regla de validación o verificación (check). Condición lógica que debe de cumplir un dato concreto para darlo por válido. Por ejemplo, una regla de validación sería restringir el campo sueldo para que siempre sea mayor de 1000. Otro ejemplo sería que la fecha de inicio de algún proceso sea anterior que la fecha de finalización.

Disparadores o triggers. Se trata de pequeños programas grabados en la base de datos que se ejecutan automáticamente cuando se cumple una determinada condición. Sirven para realizar una serie de acciones cuando ocurre un determinado evento (cuando se añade una tupla, cuando se borra un dato, cuando un usuario abre una conexión...). Los triggers permiten realizar restricciones muy potentes, pero son las más difíciles de crear.

Representación abreviada de las tablas en el modelo relacional. Las tablas se representan abreviadamente (en lugar de escribir sus tuplas) con su nombre en mayúsculas seguido del nombre de sus atributos entre paréntesis y separados por comas. Las claves primarias se subrayan, las claves candidatas se marcan con un subrayado discontinuo y las claves foráneas se indican con una flecha hacia arriba ↑.

4. Diseño lógico: paso del modelo entidad/relación al modelo relacional

En este apartado trataremos el diseño lógico de una base de datos relacional. Partiremos del resultado de la etapa del diseño conceptual expresado mediante el modelo ER y veremos cómo se puede transformar en una estructura de datos del modelo relacional.

Introducción a la transformación de entidades e interrelaciones

Los elementos básicos del modelo ER son las entidades y las interrelaciones:

- Las entidades, cuando se traducen al modelo relacional, originan tablas.
- Las interrelaciones, en cambio, cuando se transforman, pueden dar lugar a claves foráneas de alguna tabla ya obtenida o pueden dar lugar a una nueva tabla. Para poder decidir cuál es la transformación adecuada, es necesario tener en cuenta su grado y su cardinalidad:
 - Las interrelaciones binarias 1:1 y 1:N dan lugar a claves foráneas.
 - Las interrelaciones binarias M:N y todas las n-arias se traducen en nuevas tablas.

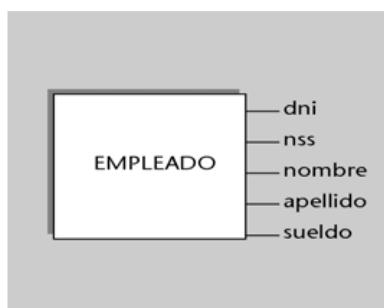
En los subapartados siguientes se explicarán de forma más concreta las transformaciones necesarias para obtener un esquema relacional a partir de un modelo ER.

A) Transformación de entidades

Como norma general, ya se ha especificado que cada entidad del modelo ER se transforma en una tabla/relación del modelo relacional. Hay alguna excepción que se comentará en los siguientes apartados.

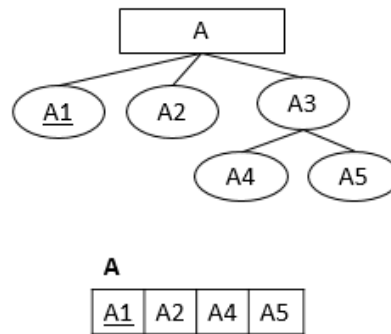
Los atributos de la entidad serán los atributos de la relación y, de forma análoga, la clave primaria de la entidad será la clave primaria de la relación.

Ejemplo de transformación de una entidad; según lo dispuesto, la entidad de la figura se transforma en la relación/tabla que tenemos a continuación:



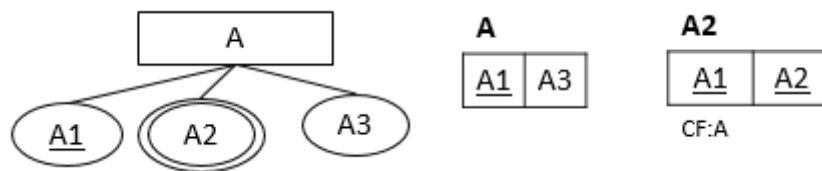
→ EMPLEADO (DNI, nss, nombre, apellido, sueldo)

Si hubiera un atributo compuesto, el atributo que ejerce la función de agrupar a los demás no formará parte de la tabla resultante en el modelo relacional. Por ejemplo:

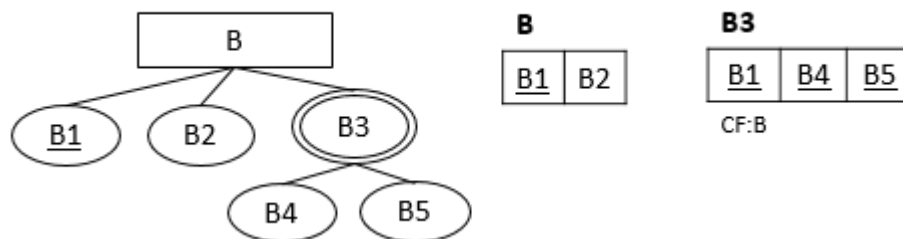


Si hubiera un atributo multivaluado, no formará parte de la tabla de la entidad a la que pertenecen. Generan una nueva tabla con la clave de la entidad y el atributo multivaluado. La clave está formada por ambos. Esta consideración se puede observar más claramente en los siguientes dos ejemplos:

Ejemplo 1:



Ejemplo 2:



Una vez transformadas todas las entidades en tablas, es preciso transformar todas las relaciones en las que intervienen estas entidades.

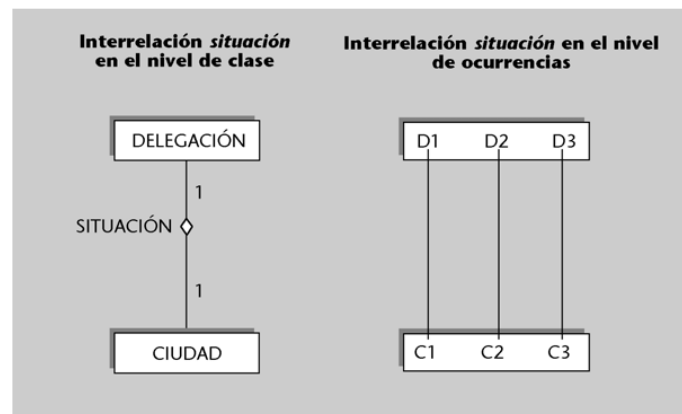
B) Transformación de relaciones binarias

Para transformar una interrelación binaria es necesario tener en cuenta su cardinalidad y si las entidades son obligatorias u opcionales en la interrelación.

B.1) Cardinalidad 1:1

De manera general, las entidades que intervienen en la interrelación 1:1 se han transformado cada una en una tabla con sus correspondientes atributos y claves. Posteriormente, para transformar la interrelación 1:1 que unía esas entidades, sólo será necesario añadir, a cualquiera de las dos relaciones/tablas que se han generado, una clave foránea que referencie a la otra relación/tabla.

Ejemplo de transformación de una interrelación binaria 1:1:



Para la interrelación de la figura anterior, tenemos dos opciones de transformación:

- Primera opción:
DELEGACIÓN (nombre-del, director, dirección, teléfono, nombre-ciudad[↑]) donde {nombre-ciudad} referencia CIUDAD
CIUDAD (nombre-ciudad, num_habit)
- Segunda opción:
DELEGACIÓN (nombre-del, director, dirección, teléfono)
CIUDAD (nombre-ciudad, num_habit, nombre-del[↑]) donde {nombre-del} referencia DELEGACIÓN

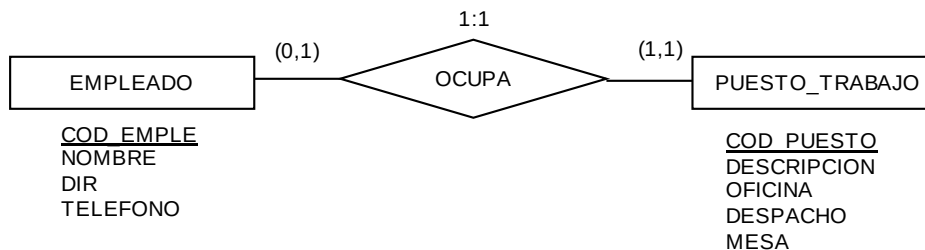
Ambas transformaciones nos permiten saber en qué ciudad hay una delegación, y qué delegación tiene una ciudad. De este modo, reflejan correctamente el significado de la interrelación situación del modelo ER.

- En la primera transformación, dado que una delegación está situada en una sola ciudad, el atributo *nombre-ciudad* tiene un único valor para cada valor de la clave primaria {*nombre-del*}. Si pudiese tener varios valores, la solución no sería correcta según la teoría relacional.
- En la segunda transformación, teniendo en cuenta que una ciudad tiene una sola delegación, el atributo *nombre-del* también toma un solo valor para cada valor de la clave primaria {*nombre-ciudad*}.

También es necesario tener en cuenta que, en las dos transformaciones, la clave foránea que se les añade se convierte en una clave alternativa de la relación porque no admite valores repetidos. Por ejemplo, en la primera transformación no puede haber más de una delegación con la misma ciudad; de este modo, *nombre-*

ciudad debe ser diferente para todas las tuplas de DELEGACIÓN. De manera análoga pasaría en la segunda transformación.

La elección de una u otra opción dependerá de la opcionalidad de la relación, intentando que haya el menor número posible de valores nulos. Veamos esta consideración de opcionalidad en un ejemplo donde las dos entidades no participan con el mismo grado de cardinalidad en una relación 1:1. En el esquema ER tendríamos (un empleado ocupa siempre un puesto de trabajo, pero un puesto de trabajo puede estar vacante):



En este caso, la opción preferente sería:

EMPLEADO (COD_EMPLE, NOMBRE, DIRECCION, TELEFONO, COD_PUESTO↑)
 PUESTOTRABAJO (COD_PUESTO, DESCRIPCION, OFICINA, DESPACHO, MESA)

Es decir, se lleva la clave principal de la entidad con cardinalidad (1,1) a la relación/tabla generada de la entidad que posee cardinalidad (0,1). La otra solución, daría en algún puesto de trabajo un valor nulo para el *cod_emple*. Por ejemplo, imaginemos que tenemos:

E1 → P1 E2 → P2 E3 → P4 El puesto P3 no lo ocupa nadie

Las opciones para este caso particular podrían ser:

- Primera opción:
 E (E1, P1, ...); (E2, P2, ...); (E3, P4...).
 P (P1, ...); (P2, ...); (P3, ...); (P4, ...).
- Segunda opción:
 E (E1, ...); (E2, ...); (E3, ...).
 P (P1, E1, ...); (P2, E2, ...); (P3, null, ...); (P4, E3, ...).

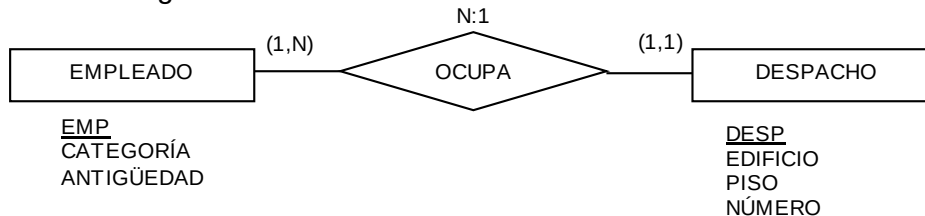
Según lo expuesto anteriormente, la segunda opción podría dar lugar a contener valores nulos, aspecto que no se da en la primera opción. Por lo tanto, la opción más adecuada sería la primera.

Una de las particularidades que se pueden presentar en este tipo de interrelación 1:1 es si las dos entidades que participan en la relación tuvieran la misma clave primaria. En este caso, ambas entidades podrían representarse en una sola tabla formada por la agregación de los atributos de los dos tipos de entidad, siendo ésta una de las excepciones de la regla que indica que a cada entidad le corresponde una tabla.

B.2) Cardinalidad 1:N

Partimos del hecho de que las entidades que intervienen en la interrelación 1:N ya se han transformado en tablas con sus correspondientes atributos. En este caso, sólo es necesario añadir en la tabla correspondiente a la entidad del lado N la clave foránea que referencie a la otra relación.

Ejemplo de transformación de una interrelación binaria 1:N: supongamos la interrelación siguiente:



Se transformaría en:

EMPLEADO (emp, categoría, antigüedad, desp[↑]) donde {desp} referencia
DESPACHO
DESPACHO (desp, edificio, piso, número)

A esta acción se la llama propagación de la clave. Esta solución nos permite saber en qué despacho está asignado cada empleado, y también nos permite consultar, para cada despacho, qué empleados hay. Es decir, refleja correctamente el significado de la interrelación ocupa. Si hubiera atributos en la relación, también se propagan hacia el lado N. Por ejemplo, tipo de silla que ocupa el empleado, se añadiría a la entidad del lado N, o sea, a empleados, y con ello a la tabla en la que se transforma.

Teniendo en cuenta que un empleado está asignado a un único despacho, el atributo *desp* tiene un valor único para cada valor de la clave primaria {*emp*}. Si hubiésemos puesto la clave foránea {*emp*} en la relación DESPACHO, la solución habría sido incorrecta, porque *emp* habría tomado varios valores, uno para cada uno de los distintos empleados que pueden estar asignados a un despacho. Imaginemos que tenemos:

E1 → D1 E2 → D2 E3 → D1

Las opciones para este caso particular podrían ser:

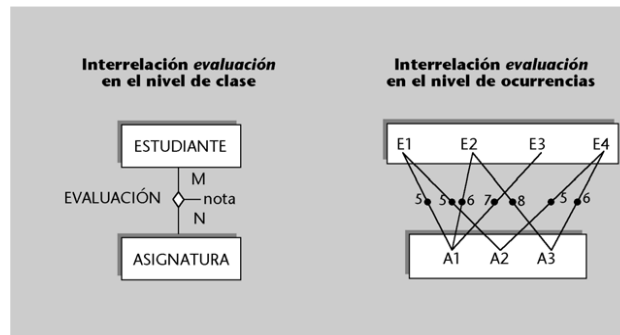
- Primera opción (correcta):
E (E1, D1, ...); (E2, D2, ...); (E3, D1...)
D (D1, ...); (D2, ...).
- Segunda opción (incorrecta):
E (E1, ...); (E2, ...); (E3, ...).
D (D1, E1-E3, ...); (D2, E2, ...). Aquí, para D1, el atributo *emp* tomaría varios valores.

Si la relación fuera opcional en el lado 1, es decir, si hubiera despachos sin ocupar, no hay ningún cambio. Pero si la opcionalidad es el lado N, es decir, si hubiera empleados que no tienen despacho, entonces en la tabla EMPLEADO se deben admitir nulos en la clave foránea *desp*.

B.3) Cardinalidad N:N

Una interrelación N:N se transforma en una relación. Su clave primaria estará formada por la unión de los atributos de la clave primaria de las dos entidades interrelacionadas. Los atributos de la interrelación serán atributos de la nueva relación.

Ejemplo de transformación de una relación binaria N:N:



La interrelación de la figura anterior se transforma en:

ESTUDIANTE (est, nombre, dirección, teléfono, fecha_nac)

ASIGNATURA (asig, horas_semana, curso)

EVALUACIÓN (est[↑], asig[↑], nota) donde {est} referencia ESTUDIANTE y {asig} referencia ASIGNATURA

Se puede observar que la clave de evaluación debe ser la unión de la clave de estudiante y la clave de asignatura para identificar completamente las parejas de la relación.

La solución presentada refleja correctamente la relación evaluación y su atributo nota. Permite saber, para cada estudiante, qué notas obtiene de las distintas asignaturas y, para cada asignatura, qué notas tienen los diferentes estudiantes de aquella asignatura.

Por lo tanto, en el caso N:N no podemos utilizar claves foráneas (propagar la clave de las entidades originales a alguna tabla en las que se transforman) para transformar la interrelación porque obtendríamos atributos que necesitarían tomar varios valores, y esto no se permite en el modelo relacional.

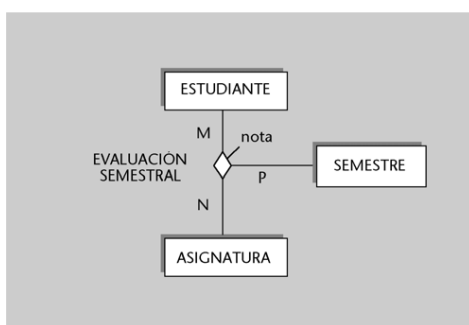
C) Transformación de interrelaciones ternarias

La transformación de las interrelaciones ternarias presenta similitudes importantes con la transformación de las binarias N:N. No es posible representar la interrelación mediante claves foráneas, sino que es necesario usar una nueva tabla. Para que la nueva relación refleje toda la información que modeliza la interrelación, es necesario que contenga las claves primarias de las tres entidades interrelacionadas y los atributos de la interrelación. La clave primaria de la nueva relación depende de la cardinalidad de la interrelación.

C.1) Cardinalidad N:N:N

Cuando la cardinalidad de la interrelación es N:N:N, la relación que se obtiene de su transformación tiene como clave primaria todos los atributos que forman las claves primarias de las tres entidades interrelacionadas.

Ejemplo de transformación de una interrelación ternaria N:N:N:



La interrelación anterior se transforma en:

ESTUDIANTE (est, ...)
ASIGNATURA (asig, ...)
SEMESTRE (sem, ...)
EVALUACIÓN-SEMESTRAL (est↑, asig↑, sem↑, nota) donde {est}
referencia ESTUDIANTE, {asig} referencia ASIGNATURA y {sem}
referencia SEMESTRE

Para identificar completamente cada tupla de la tabla, la clave debe constar de la clave de estudiante, de la clave de asignatura y de la clave de semestre. Si nos faltase una de las tres, la clave de la relación podría tener valores repetidos. Consideremos, por ejemplo, que no tuviésemos la clave de semestre. Dado que semestre está conectado con “muchos” en la interrelación, puede haber estudiantes que han sido evaluados de una misma asignatura en más de un semestre. Entonces, para estos casos habría valores repetidos en la clave de la relación EVALUACION-SEMESTRAL, lo que no permitiría identificar unívocamente cada elemento de la relación. Lo vemos sobre un ejemplo:

E1, A1 → S1 E1, A1 → S2 E1, A1 → S3

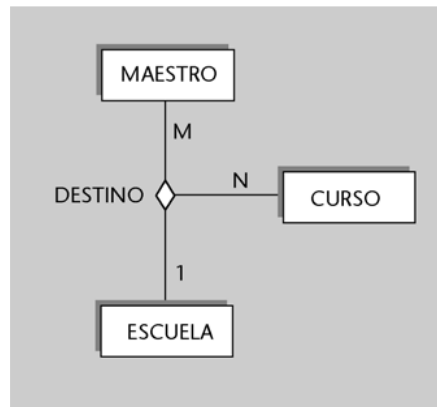
En la tabla EVALUACIÓN-SEMESTRAL, si la clave sólo estuviera formada por *est* y *asig*, quedaría: E-V (E1, A1, S1); : E-V (E1, A1, S2); : E-V (E1, A1, S3). Sin embargo, si la clave estuviera formada por los tres atributos clave de las entidades, quedaría: E-V (E1, A1, S1,...); (E1, A1, S2,...); (E1, A1, S3,...). De esta última manera, sí se podría identificar unívocamente cada elemento de la relación.

Del mismo modo que es necesaria la clave de semestre, también lo son la de estudiante y la de asignatura.

C.2) Cardinalidad N:N:1

Cuando la conectividad de la interrelación es N:N:1, la tabla que se obtiene de su transformación tiene como clave primaria todos los atributos que forman las claves primarias de las dos entidades de los lados de la interrelación etiquetados con N.

Ejemplo de transformación de una interrelación ternaria N:N:1:



Esta relación refleja los destinos que se dan a los maestros de escuela en los diferentes cursos. El 1 que figura en el lado de escuela significa que un maestro no puede ser destinado a más de una escuela en un mismo curso.

El ejemplo de la figura se transforma en:

MAESTRO (código-maestro, ...)

CURSO (código-curso, ...)

ESCUELA (código-esc, ...)

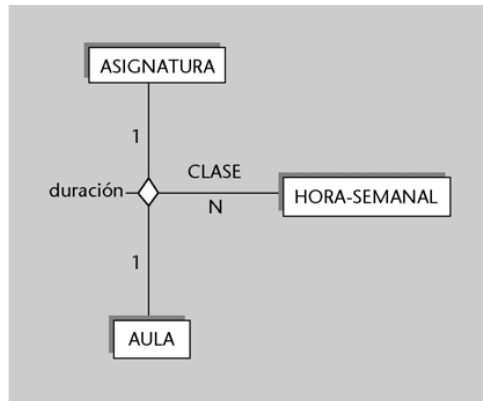
DESTINO (código-maestro[↑], código-curso[↑], código-esc[↑]) donde {código-maestro} referencia MAESTRO {código-curso} referencia CURSO y {código-esc} referencia ESCUELA

No es necesario que la clave incluya código-esc para identificar completamente la relación. Si se fijan, para un maestro y un curso, no puede haber más de una escuela de destino y, por lo tanto, no habrá claves repetidas.

C.3) Cardinalidad N:1:1

Cuando la cardinalidad de la relación es N:1:1, la tabla que se obtiene de su transformación tiene como clave primaria los atributos que forman la clave primaria de la entidad del lado N y los atributos que forman la clave primaria de cualquiera de las dos entidades que están conectadas con 1. Así pues, hay dos posibles claves para la tabla que se obtiene. Son dos claves candidatas entre las cuales el diseñador deberá escoger la primaria.

Ejemplo de transformación de una interrelación ternaria N:1:1, habiendo dos posibilidades:



- 1ª opción:

HORA-SEMANTAL (código-hora, ...)

AULA (código-aula, ...)

ASIGNATURA (asig, ...)

CLASE (código-hora↑, código-aula↑, asig↑, duración) donde {código-hora} referencia HORA-SEMANTAL, {código-aula} referencia AULA y {asig} referencia ASIGNATURA

En este caso, la clave, a pesar de no incluir el atributo asig en la clave, identifica completamente la relación porque para una hora-semanal y un aula determinadas, hay una única asignatura de la que se imparte clase.

Considerando la cardinalidad (N:1:1):

- AU → A1 AS → Mate HS → 1ª; HS → 2ª; HS → 3ª...
- En el A1 Mate se puede impartir a 1ª un día, a 3ª otro día y a 2ª otro día.
- A 1ª hora en el A1 se imparte Mate.
- A 1ª hora Mate se imparte en el A1.

CLASE (1ª, A1, Mate...); (2ª, A1, Mate); (3ª, A1, Mate).

- 2ª opción:

HORA-SEMANTAL (código-hora, ...)

AULA (código-aula, ...)

ASIGNATURA (asig, ...)

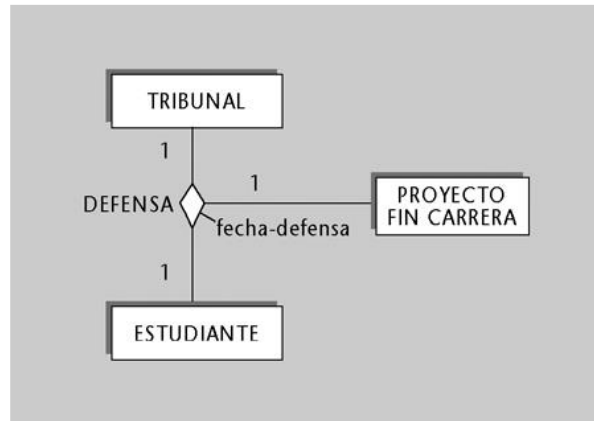
CLASE (código-hora↑, asig↑, código-aula↑, duración) donde {código-hora} referencia HORA-SEMANTAL, {código-aula} referencia AULA y {asig} referencia ASIGNATURA

Ahora la clave incluye el atributo asig y, en cambio, no incluye el atributo código-aula. La tabla también queda completamente identificada porque, para una asignatura y hora-semanal determinadas, de aquella asignatura se da clase en una sola aula a aquella hora.

C.4) Cardinalidad 1:1:1

Cuando la cardinalidad de la interrelación es 1:1:1, la relación que se obtiene de su transformación tiene como clave primaria los atributos que forman la clave primaria de dos entidades cualesquiera de las tres interrelacionadas. Así pues, hay tres claves candidatas para la relación entre las cuales el diseñador deberá escoger la primaria.

Ejemplo de transformación de una interrelación ternaria 1:1:1:



Esta interrelación registra información de defensas de proyectos de fin de carrera. Intervienen en ella el estudiante que presenta el proyecto, el proyecto presentado y el tribunal evaluador.

La transformación del ejemplo anterior se muestra a continuación:

TRIBUNAL (trib, ...)

ESTUDIANTE (est, ...)

PROYECTO-FIN-CARRERA (pro, ...)

Para la nueva relación DEFENSA, tenemos las tres posibilidades siguientes:

• 1ª opción:

DEFENSA (trib[↑], est[↑], pro[↑], fecha-defensa) donde {trib} referencia TRIBUNAL, {est} referencia ESTUDIANTE y {pro} referencia PROYECTO-FIN-CARRERA

• 2ª opción:

DEFENSA (pro[↑], est[↑], trib[↑], fecha-defensa) donde {trib} referencia TRIBUNAL, {est} referencia ESTUDIANTE y {pro} referencia PROYECTO-FIN-CARRERA

• 3ª opción:

DEFENSA (pro[↑], trib[↑], est[↑], fecha-defensa) donde {trib} referencia TRIBUNAL, {est} referencia ESTUDIANTE y {pro} referencia PROYECTO-FIN-CARRERA

En los tres casos, es posible comprobar que la clave identifica completamente la relación si se tiene en cuenta la conectividad de la interrelación defensa.

D) Transformación de relaciones n-arias

La transformación de las relaciones n-arias se puede ver como una generalización de lo que hemos explicado para las ternarias.

En todos los casos, la transformación de una relación n-aria consistirá en la obtención de una nueva tabla que contiene todos los atributos que forman las claves primarias de las n entidades interrelacionadas y todos los atributos de la relación.

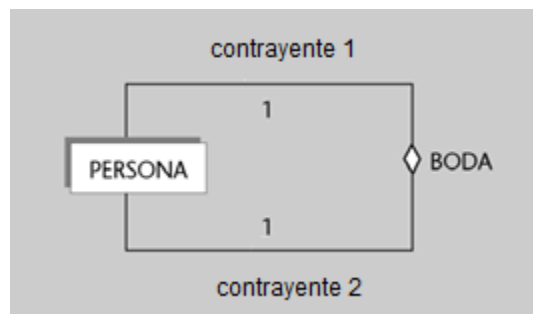
Podemos distinguir los casos siguientes:

- a) Si todas las entidades están conectadas con “muchos”, la clave primaria de la nueva relación estará formada por todos los atributos que forman las claves de las n entidades interrelacionadas.
- b) Si una o más entidades están conectadas con “uno”, la clave primaria de la nueva relación estará formada por las claves de $n-1$ de las entidades interrelacionadas, con la condición de que la entidad, cuya clave no se ha incluido, debe ser una de las que está conectada con “uno”.

E) Transformación de relaciones recursivas

Las transformaciones de las relaciones recursivas son similares a las que hemos visto para el resto de las interrelaciones. De este modo, si una interrelación recursiva tiene conectividad 1:1 o 1:N, da lugar a una clave foránea, y si tiene conectividad N:N o es n-aria, origina una nueva tabla. Mostraremos la transformación de algunos ejemplos concretos de interrelaciones recursivas para ilustrar los detalles de esta afirmación.

Ejemplo de transformación de una interrelación recursiva binaria 1:1:

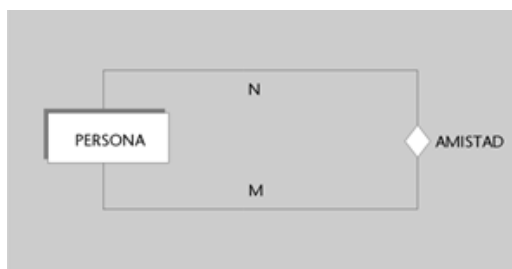


Las interrelaciones 1:1 originan una clave foránea que se pone en la tabla correspondiente a una de las entidades interrelacionadas. En nuestro ejemplo, sólo hay una entidad interrelacionada, la entidad persona. Entonces, la clave foránea deberá estar en la tabla PERSONA. Esta clave foránea deberá referenciar a la misma tabla para que refleje una interrelación entre una ocurrencia de persona y otra ocurrencia de persona. Así, obtendremos:

PERSONA (código-per, ..., código-conyuge[↑]) donde {código-conyuge} referencia PERSONA y código-conyuge admite valores nulos (en el caso de que esa persona no haya celebrado boda).

Conviene tener en cuenta que, en casos como éste, los atributos de la clave foránea no pueden tener los mismos nombres que los atributos de la clave primaria que referencian porque, según la teoría relacional, una relación no puede tener nombres de atributos repetidos.

Ejemplo de transformación de una interrelación recursiva N:N:



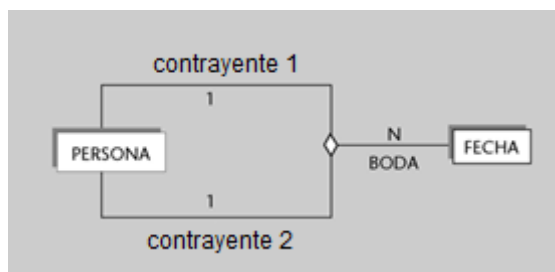
Las relaciones N:N se traducen en nuevas tablas que tienen como clave primaria las claves de las entidades relacionadas. En nuestro ejemplo, la relación vincula ocurrencias de persona con otras ocurrencias de persona. En este caso, la clave primaria de la nueva tabla estará formada por la clave de la entidad persona dos veces. Convendrá dar nombres diferentes a todos los atributos de la nueva tabla.

De este modo, la traducción del ejemplo anterior será:

PERSONA (código-per, ...)

AMISTAD (código-per[↑], código-per-amiga[↑]) donde {código-per} referencia PERSONA y {código-per-amiga} referencia PERSONA

Ejemplo de transformación de una interrelación recursiva n-aria N:1:1:



La anterior relación boda es recursiva, ternaria y tiene conectividad N:1:1. Las interrelaciones N:1:1 originan siempre una nueva tabla que contiene, además de los atributos de la relación, todos los atributos que forman la clave primaria de las tres entidades interrelacionadas.

En nuestro ejemplo, la relación asocia ocurrencias de persona con otras ocurrencias de persona y con ocurrencias de fecha. Entonces, la clave de persona tendrá que figurar dos veces en la nueva tabla, y la clave de fecha, solo una. La clave primaria de la relación que se obtiene para relaciones N:1:1 está formada por la clave de la entidad del lado N y por la clave de una de las entidades de los lados 1. Con ello, en los dos lados 1 de la relación tenemos la misma entidad: persona. La clave primaria estará formada por la clave de la entidad fecha y por la clave de la entidad persona.

Según todo esto, la transformación será la siguiente:

PERSONA (código-per, ...)

FECHA (fecha-bod, ...)

BODA (fecha-bod[↑], código-per[↑], código-conyuge[↑]) donde {fecha-bod} referencia FECHA, {código-per} referencia PERSONA y {código-conyuge} referencia PERSONA

F) Transformación de entidades débiles

Las entidades débiles se traducen al modelo relacional igual que el resto de entidades, con una pequeña diferencia. Estas entidades siempre están en el lado N de una interrelación 1:N que completa su identificación.

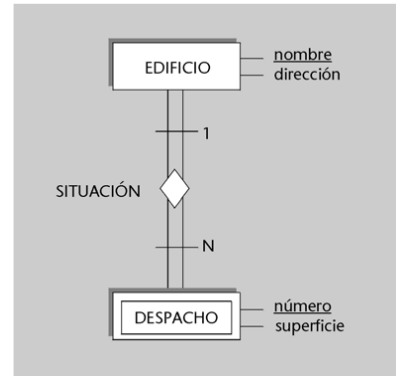
Así pues, la clave foránea originada por esta interrelación 1:N debe formar parte de la clave primaria de la relación correspondiente a la entidad débil. Si fuera un caso especial de débil con interrelación 1:1 se transformaría igual que las 1:1 genéricas, mediante el mecanismo de propagación de claves ya estudiado.

Ejemplo de transformación de entidad débil:

EDIFICIO (nombre, dirección)

DESPACHO (nombre[↑], número, superficie) donde
{nombre} referencia EDIFICIO

La clave foránea {*nombre*} forma parte también de la clave primaria de DESPACHO. Si no fuese así, y la clave primaria contuviese sólo el atributo número, los despachos no quedarían totalmente identificados, teniendo en cuenta que puede haber despachos situados en edificios diferentes que tengan el mismo número.



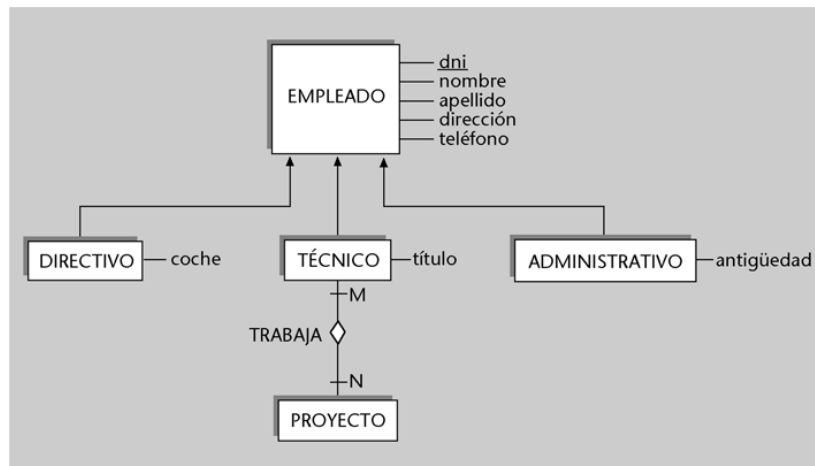
G) Transformación de la generalización/especialización

Hay varias opciones:

- 1) Integrar todas las entidades en una única tabla, eliminando los subtipos. Esta nueva entidad contendrá todos los atributos del supertipo, todos los de los subtipos y los que hagan falta para discernir a qué tipo de subclase pertenece cada atributo. Todas las relaciones se mantienen con la nueva entidad. La gran ventaja es la simplicidad. El gran inconveniente es la gran cantidad de valores nulos que se origina y la pérdida de significado. Si las subclases se diferencian en muy pocos atributos y si las relaciones que las asocian con el resto de entidades del esquema son las mismas para todas las subclases, esta opción puede ser la más adecuada.
- 2) Eliminación del supertipo. Se diseñan tantas tablas como subclases con sus atributos y claves primarias respectivas y se les añaden los atributos de la superclase. Las relaciones con el supertipo las heredan los subtipos. Como inconvenientes tiene que se crea redundancia al repetir los atributos del supertipo y que el número de relaciones aumenta. Esta opción suele ser la que pierde más semántica ya que si la relación es parcial, ¿dónde están las entidades de la superclase que no pertenecen a ningún subtipo especificado?; y si es solapada, habrá atributos redundantes. Por ello, esta opción sólo tendría sentido aplicarse para jerarquías totales y exclusivas o cuando existen muchos atributos distintos entre los subtipos y los accesos realizados sobre los datos de los distintos subtipos siempre afectan a atributos comunes.
- 3) Cada una de las entidades superclase y subclase que forman parte de una generalización/especialización se transforman en tablas:
 - a) La tabla de la entidad superclase tiene como clave primaria la clave de la entidad superclase y contiene todos los atributos comunes. Puede incluir un atributo discriminador.

b) Las tablas de las entidades subclase tienen como clave primaria la clave de la entidad superclase y contienen los atributos específicos de la subclase. Esta opción suele ser, en términos generales, la menos eficiente, pero puede ser buena opción cuando existen muchos atributos distintos entre los subtipos y se quieren mantener de todas las maneras los atributos comunes a todos ellos en una relación.

Veamos un ejemplo que contiene una generalización/especialización y también una interrelación N:N en la que interviene una de las entidades subclase.



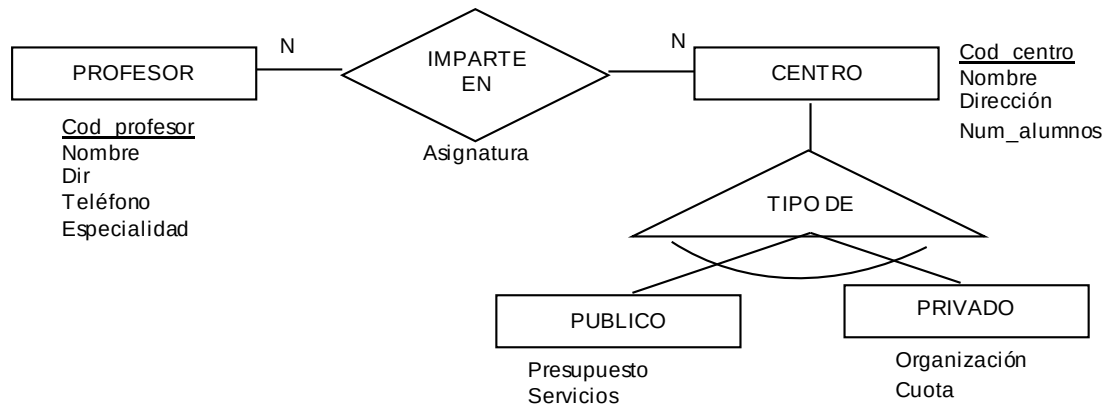
Este ejemplo se traduce al modelo relacional como se indica a continuación:

EMPLEADO (DNI, nombre, apellido, dirección, teléfono)
 DIRECTIVO (DNI[↑], coche) donde {DNI} referencia EMPLEADO
 ADMINISTRATIVO (DNI[↑], antigüedad) donde {DNI} referencia EMPLEADO
 TÉCNICO (DNI[↑], título) donde {DNI} referencia EMPLEADO
 PROYECTO (pro, ...)
 TRABAJA (DNI[↑], pro[↑], superficie) donde {DNI} referencia TÉCNICO y {pro} referencia PROYECTO

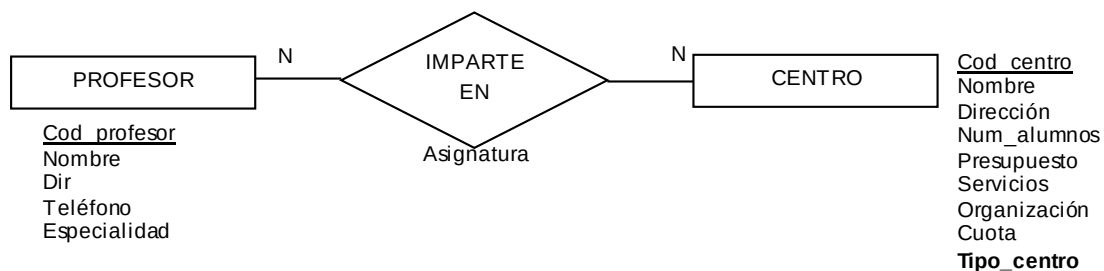
Conviene observar que los atributos comunes se han situado en la relación EMPLEADO y que los atributos específicos se han situado en la relación de su entidad subclase. De este modo, coche está en DIRECTIVO, título en TÉCNICO y antigüedad en ADMINISTRATIVO.

Por otro lado, la interrelación entre los empleados técnicos y los proyectos denominada trabaja se transforma en la relación TRABAJA. Observad que esta relación tiene una clave foránea que referencia sólo a los empleados técnicos, y no a los empleados directivos o administrativos.

Veamos otro ejemplo que refleja generalizaciones/especializaciones:



1ª opción: Una sola entidad para la superclase y las subclases.



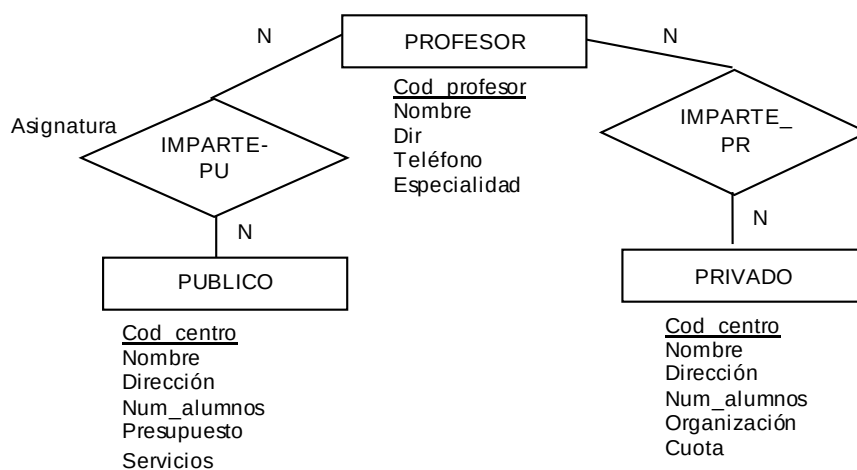
CENTRO (Cod_centro, Nombre, Dirección, Num_alumnos, Presupuesto, Servicios, Organización, Cuota, Tipo_centro)

PROFESOR (Cod_profesor, Nombre, Dir, Teléfono, Especialidad)

IMPARTE (Cod_profesor↑, Cod_centro↑, Asignatura)

2ª opción:

Las subclases producen tablas que heredan los atributos de la superclase.



PROFESOR (Cod_profesor, Dirección, Nombre, Teléfono, Especialidad)

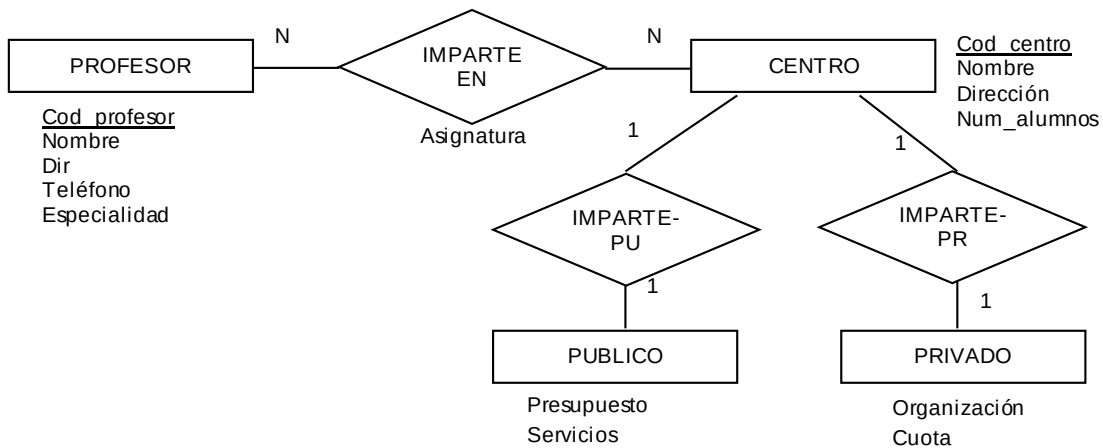
PÚBLICO (Cod_centro, Nombre, Dirección, Num_alumnos, Servicios, Presupuesto)

PRIVADO (Cod_centro, Nombre, Dirección, Num_alumnos, Organización, Cuota)

IMPARTE_PU (Cod_profesor↑, Cod_centro↑, Asignatura)

IMPARTE_PR (Cod_profesor↑, Cod_centro↑, Asignatura)

3ª opción: Considerar la tabla de la entidad superclase Centro y las de las subclases Centro-Público y Centro-Privado.



PROFESOR (Cod_profesor, Dirección, Nombre, Teléfono, Especialidad)

CENTRO (Cod_centro, Nombre, Dirección, Num_alumnos, Tipo_centro)

PÚBLICO (Cod_centro, Servicios, Presupuesto)

PRIVADO (Cod_centro, Organización, Cuota)

IMPARTE (Cod_profesor↑, Cod_centro↑, Asignatura)

5. Ejemplo completo de paso del modelo entidad/relación al modelo relacional

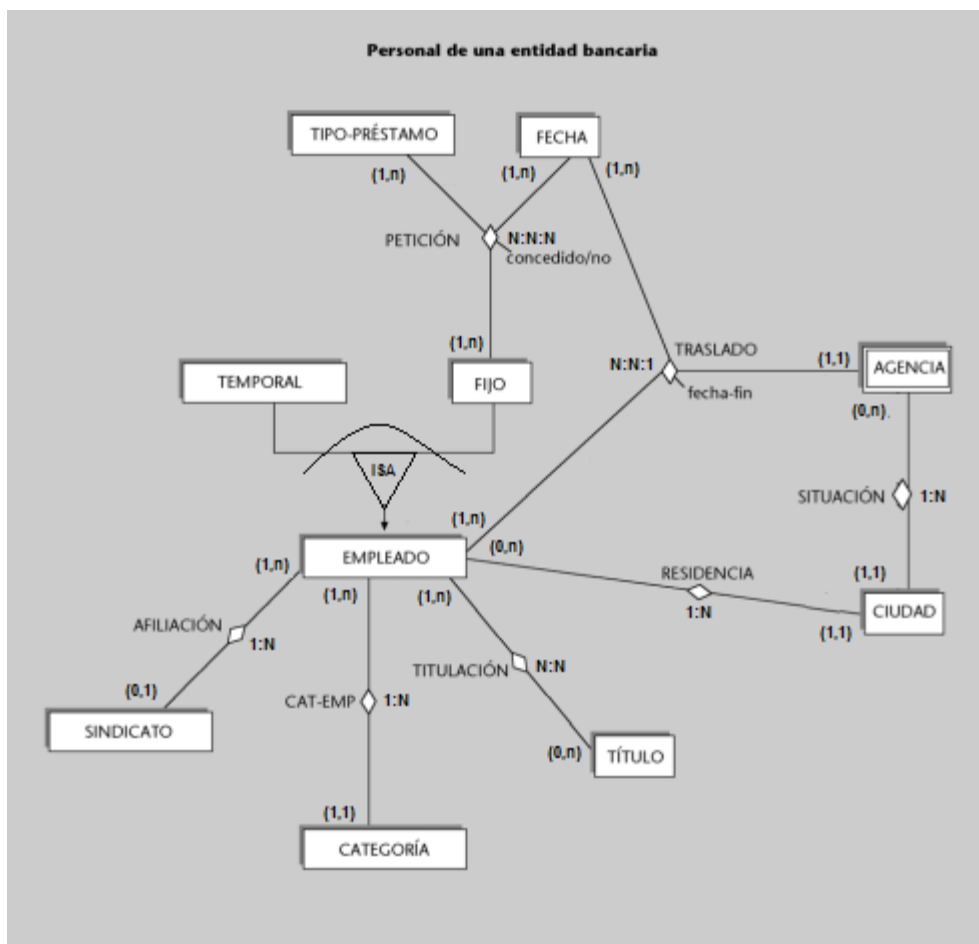
Diseñar una base de datos para la gestión del personal de una entidad bancaria determinada que dispone de muchos empleados y de una amplia red de agencias.

A continuación, se describen los requisitos de los usuarios:

- Los empleados se identifican por un código de empleado. También deseamos conocer su DNI, su NSS, su nombre y su apellido. Es importante registrar su ciudad de residencia, considerando que hay ciudades donde no reside ningún empleado.
- Interesa saber las ciudades donde se ubican las diversas agencias de la entidad bancaria. Estas agencias bancarias se identifican por la ciudad donde están y por un nombre que permite distinguir las agencias de una misma ciudad. Se quiere tener constancia del número de habitantes de las ciudades, así como de la dirección y el número de teléfono de las agencias. Se debe considerar que la base de datos también incluye ciudades donde no hay ninguna agencia.
- Un empleado, en un momento determinado, trabaja en una sola agencia, lo cual no impide que pueda ser trasladado a otra o, incluso, que vuelva a trabajar en una agencia donde ya había trabajado anteriormente. Se quiere tener constancia del historial del paso de los empleados por las agencias.
- Los empleados pueden tener títulos académicos (aunque no todos los tienen), que debemos recoger.
- Cada empleado tiene una categoría laboral determinada (auxiliar, oficial de segunda, oficial de primera, etc.). A cada categoría le corresponde un sueldo base determinado y un precio por hora extra también determinado.

- f) Algunos empleados (no todos) están afiliados a alguna central sindical. Se ha llegado al pacto de descontar de la nómina mensual la cuota sindical a los afiliados a cada central. Esta cuota es única para todos los afiliados a una central determinada. Es necesario almacenar las afiliaciones de los empleados y las cuotas sindicales.
- g) Hay dos tipos de empleados diferentes:
- Los que tienen contrato fijo, cuya antigüedad queremos conocer.
 - Los que tienen contrato temporal, de los cuales nos interesa saber las fechas de inicio y finalización de su último contrato. Si un empleado temporal pasa a ser fijo, se le asigna un nuevo código de empleado; consideraremos que un empleado fijo nunca pasa a ser temporal. Todo lo que se ha indicado hasta ahora (traslados, categorías, afiliación sindical, etc.) es aplicable tanto a empleados fijos como a temporales.
- h) Los empleados fijos tienen la posibilidad de pedir diferentes tipos preestablecidos de préstamos (por matrimonio, por adquisición de vivienda, por estudios, etc.), que pueden ser concedidos o no. No hay ninguna limitación a la hora de pedir varios préstamos a la vez, siempre que no se pida más de uno del mismo tipo al mismo tiempo. Se quiere registrar los préstamos pedidos por los empleados, y hacer constar si han sido concedidos o no. Cada tipo de préstamo tiene establecidas diferentes condiciones: el tipo de interés y el periodo de vigencia del préstamo.

La figura muestra un diagrama E-R que satisface los requisitos anteriores:



Los atributos de las entidades del diagrama son los siguientes (las claves primarias se han subrayado):

- EMPLEADO (código-empleado, dni, nss, nombre, apellido)
- FIJO (código-empleado, antigüedad)
- TEMPORAL (código-empleado, fecha-inicio-contrato, fecha-final-contrato)
- CIUDAD (nombre-ciudad, número-habitantes)
- AGENCIA (nombre-agencia, dirección, teléfono) entidad débil: nombre-agencia la identifica parcialmente, se identifica completamente con la ciudad de situación
- TÍTULO (nombre-título)
- CATEGORÍA (nombre-categoría, sueldo-base, hora-extra)
- SINDICATO (central, cuota)
- TIPO-PRÉSTAMO (código-préstamo, tipo-interés, período-vigencia)
- FECHA (fecha)

Comentaremos los aspectos más complejos de este modelo E-R:

- 1) La entidad Agencia se ha considerado una entidad débil porque su atributo nombre-agencia sólo permite distinguir las agencias situadas en una misma ciudad, pero para identificar de forma total una agencia, es necesario saber en qué ciudad está situada.
- 2) La relación Petición es ternaria y asocia a empleados fijos que hacen peticiones de préstamos, tipos de préstamos pedidos por los empleados y fechas en las que se hacen estas peticiones.
 - El lado de la entidad fecha se conecta con “muchos” porque un mismo empleado puede pedir un mismo tipo de préstamo varias veces en fechas distintas. La entidad Fijo se conecta con “muchos” porque un tipo de préstamo determinado puede ser pedido en una misma fecha por varios empleados. También la entidad Tipo-préstamo se conecta con “muchos” porque es posible que un empleado en una fecha determinada pida más de un préstamo de tipo diferente.
 - El atributo concedido indica si el préstamo se ha concedido o no. Es un atributo de la relación porque su valor depende del empleado fijo que hace la petición, del tipo de préstamo pedido y de la fecha de petición.
- 3) La relación Traslado también es ternaria y permite registrar el paso de los empleados por las distintas agencias. Un traslado concreto asocia a un empleado, una agencia donde él trabajará y una fecha inicial en la que empieza a trabajar en la agencia.
 - El atributo de la relación fecha-fin indica en qué fecha finaliza su asignación a la agencia (fecha-fin tendrá el valor nulo cuando un empleado trabaja en una agencia en el momento actual y no se sabe cuándo se le trasladará).
 - Esta relación no registra todas y cada una de las fechas en las que un empleado está asignado a una agencia, sino sólo la fecha inicial y la fecha final de la asignación. Es muy habitual que, para informaciones que son ciertas durante todo un periodo de tiempo, se registre en la base de datos únicamente el inicio y el final del periodo.

- La entidad Agencia se ha conectado con “uno” en la relación traslado, porque no puede ocurrir que, en una fecha, un empleado determinado sea trasladado a más de una agencia.
- 4) Finalmente, comentaremos la especialización de la entidad Empleado. Los empleados pueden ser de dos tipos; se quieren registrar propiedades diferentes para cada uno de los tipos y también se requieren algunas propiedades comunes a todos los empleados.

A continuación, se realizarán las transformaciones al modelo relacional.

Se empieza por transformar las entidades en tablas y todas las relaciones 1:1 y 1:N en claves externas.

- EMPLEADO (código-empleado, DNI, nss, nombre, apellido, nombrecateg[↑], central[↑], ciudad-res[↑]) donde {nombre-categoría} referencia Categoría, {ciudad-res} referencia Ciudad y {central} referencia Sindicato, este atributo admite valores nulos.
- FIJO (código-empleado[↑], antigüedad) donde {código-empleado} referencia Empleado
- TEMPORAL (código-empleado[↑], fecha-inicio-contrato, fecha-final-contrato) donde {código-empleado} referencia Empleado
- CIUDAD (nombre-ciudad, número-habitantes)
- AGENCIA (nombre-ciudad[↑], nombre-agencia, dirección, teléfono) donde {nombre-ciudad} referencia Ciudad
- TÍTULO (nombre-título)
- CATEGORÍA (nombre-categoría, sueldo-base, hora-extra)
- SINDICATO (central, cuota)
- TIPO-PRÉSTAMO (código-préstamo, tipo-interés, período-vigencia)
- FECHA (fecha)

Se ha transformado la especialización de la entidad Empleado según la tercera opción. En la tabla Agencia, el atributo nombre-ciudad es una clave foránea y además forma parte de la clave primaria.

A continuación, se generan las tablas que se obtienen de las relaciones:

- TITULACIÓN (código-empleado[↑], nombre-título[↑]) donde {código-empleado} referencia Empleado y {nombre-título} Título
- TRASLADO (código-empleado[↑], fecha[↑], nombre-ciudad[↑], nombre-agencia[↑], fecha-fin) donde {código-empleado} referencia Empleado, {nombre-ciudad, nombre-agencia} referencia Agencia y {fecha} referencia Fecha
- PETICIÓN (código-empleado[↑], código-préstamo[↑], fecha[↑], concedido) donde {código-empleado} referencia Fijo, {código-préstamo} referencia Tipo-Préstamo y {fecha} referencia Fecha

6. Anexo: las 12 reglas de Codd

Preocupado por los productos que decían ser sistemas gestores de bases de datos relacionales (RDBMS) sin serlo, Codd publica las 12 reglas que debe cumplir todo DBMS para ser considerado relacional. Estas reglas en la práctica las cumplen pocos sistemas relacionales. Las reglas son:

- (1) Información. Toda la información de la base de datos (metadatos) debe estar representada explícitamente en el esquema lógico. Es decir, todos los datos están en las tablas.
- (2) Acceso garantizado. Todo dato es accesible sabiendo la tabla, el valor de su clave y el nombre de la columna o atributo que contiene el dato.
- (3) Tratamiento sistemático de los valores nulos. El DBMS debe permitir el tratamiento adecuado de estos valores. De ese modo, el valor nulo se utiliza para representar la ausencia de información de un determinado registro en un atributo concreto.
- (4) Catálogo en línea basado en el modelo relacional. Los metadatos deben de ser accesibles usando un esquema relacional. Es decir, la forma de acceder a los metadatos es la misma que la de acceder a los datos.
- (5) Sublenguaje de datos completo. Al menos debe de existir un lenguaje que permita el manejo completo de la base de datos. Este lenguaje, por lo tanto, debe permitir realizar cualquier operación sobre la misma.
- (6) Actualización de vistas. El SGBD debe encargarse de que las vistas muestren la última información. No son válidas vistas que muestren datos que no están al día.
- (7) Inserciones, modificaciones y eliminaciones de alto nivel. Cualquier operación de modificación debe actuar sobre conjuntos de filas o registros, nunca deben actuar registro a registro.
- (8) Independencia física. Los datos deben de ser accesibles desde la lógica de la base de datos aun cuando se modifique el almacenamiento. La forma de acceder a los datos no varía porque el esquema físico de la base de datos, cambie.
- (9) Independencia lógica. Los programas no deben verse afectados por cambios en las tablas. Que las tablas cambien no implica que cambien los programas.
- (10) Independencia de integridad. Las reglas de integridad deben almacenarse en la base de datos (en el diccionario de datos), no en los programas de aplicación.
- (11) Independencia de la distribución. El sublenguaje de datos debe permitir que sus instrucciones funcionen igualmente en una base de datos distribuida que en una que no lo es.
- (12) No subversión. Si el SGBD posee un lenguaje procedimental que permita crear bucles de recorrido fila a fila, éste no puede utilizarse para incumplir o evitar las reglas relacionales anteriores. Especialmente la regla 7 no puede ser incumplida por ningún lenguaje del SGBD.